



WELCOME





C++ Operators, Type conversion & Conditional Statements

Agenda

C++ Bridge Course



Operators



v/s



Logical Operators

Logical Operators

<i>Operator</i>	<i>Python</i>	<i>C++</i>
<i>AND</i>	<i>and</i>	<i>&&</i>
<i>OR</i>	<i>or</i>	<i> </i>
<i>NOT</i>	<i>not</i>	<i>!</i>

Operators

Logical Operators

C++ 

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    bool x = true;
    bool y = false;
    cout << (x && y) << endl;
    return 0;
}
```

Output 

0

Python 

```
x = True
y = False
print(x and y)
```

Output 

False

Arithmetic Operators

Arithmetic Operators

<i>Operator</i>	<i>Python</i>	<i>C++</i>
<i>Addition</i>	+	+
<i>Subtraction</i>	-	-
<i>Multiplication</i>	*	*
<i>Division</i>	/	/
<i>Modulus</i>	%	%

Relational Operators

Relational Operators

<i>Operator</i>	<i>Python</i>	<i>C++</i>
<i>Greater than</i>	>	>
<i>Less than</i>	<	<
<i>Not equal</i>	!=	!=
<i>Equal</i>	==	==
<i>Greater than or equal</i>	>=	>=
<i>Less than or equal</i>	<=	<=

Assignment Operators

Assignment Operators

<i>Operator</i>	<i>Python</i>	<i>C++</i>
<i>Assignment</i>	=	=
<i>Add and assign</i>	+=	+=
<i>Subtract and assign</i>	-=	-=
<i>Multiply and assign</i>	*=	*=
<i>Divide and assign</i>	/=	/=
<i>Modulus and assign</i>	%=	%=

Key Differences

Exponent Operator:

- **Python:** e.g., `2 ** 3` for 2^3 .
- **C++:** No exponent operator.
Use `pow()` from `<cmath>` (e.g., `pow(2, 3)`).

Key Differences

Logical Operators

- Python uses **and**, **or**, **not**.
- C++ uses **&&**, **||**, **!**

Key Differences

Output for Boolean

- **Python**: Prints **True** or **False**.
- **C++**: Prints **1** or **0**.

Quiz Time!



Type Conversion



C++

Type Conversion

The process of converting the value of one data type (**int**, **char**, **float**, **etc.**) to another data type.



C++

Type Conversion in C++

C++ provides two major types of type conversions for **primitive** data types:

- Explicit Type Conversion (Type Casting)
- Implicit Type Conversion



C++

Type Conversion in C++

C++ provides two major types of type conversions for **primitive** data types:



Explicit Type Conversion (Type Casting)

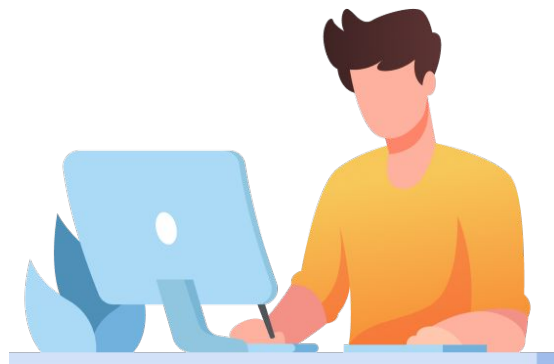
→ Implicit Type Conversion



C++

Explicit Type Conversion

- ✓ In Explicit Type Conversion, **programmers** explicitly **change** the **data type** of a value to the desired data type. This type of conversion is also called **Type Casting**.



Explicit Type Conversion

double → int

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    double x = 5.6;
    int y = (int)x;
    cout << y << endl;
    return 0;
}
```

*After conversion, the decimal portion of the double value is **trimmed**, which results in **y** value **5**.*

Explicit Type Conversion

double → int

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    double x = 5.6;
    int y = (int)x;
    cout << y << endl;
return 0;
}
```

Output 

5

Explicit Type Conversion

char → int

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    char c = 'A';
    int d = (int)c;
    cout << d << endl;
    return 0;
}
```

*A character returns its ASCII value
when converted into an integer.
Here, 'A' is 65 in ASCII.*

Explicit Type Conversion

ASCII Table

Dec	Hex	Char
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9

Dec	Hex	Char	78	4E	N
65	41	A	79	4F	O
66	42	B	80	50	P
67	43	C	81	51	Q
68	44	D	82	52	R
69	45	E	83	53	S
70	46	F	84	54	T
71	47	G	85	55	U
72	48	H	86	56	V
73	49	I	87	57	W
74	4A	J	88	58	X
75	4B	K	89	59	Y
76	4C	L	90	5A	Z
77	4D	M			

Dec	Hex	Char	110	6E	n
97	61	a	111	6F	o
98	62	b	112	70	p
99	63	c	113	71	q
100	64	d	114	72	r
101	65	e	115	73	s
102	66	f	116	74	t
103	67	g	117	75	u
104	68	h	118	76	v
105	69	i	119	77	w
106	6A	j	120	78	x
107	6B	k	121	79	y
108	6C	l	122	7A	z
109	6D	m			

Explicit Type Conversion

char → int

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    char c = 'A';
    int d = (int)c;
    cout << d << endl;
    return 0;
}
```

Output 

65

Explicit Type Conversion

int → char

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int m = 65;
    char n = (char)m;
    cout << n << endl;
    return 0;
}
```

*The integer **65** is converted into a character, which corresponds to 'A' in **ASCII**.*

Explicit Type Conversion

int → char

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int m = 65;
    char n = (char)m;
    cout << n << endl;
    return 0;
}
```

Output 

A

C++

Type Conversion in C++

C++ provides two major types of type conversions for **primitive** data types:

✓ **Explicit Type Conversion (Type Casting)**



Implicit Type Conversion



C++

Implicit Type Conversion

- ✓ The C++ compiler **automatically** converts one primitive data type to another when a value is assigned to a variable of a different type. This process is called **Implicit Type Conversion**.



```
int x = 10;  
long long y = x;
```

C++

Implicit Type Conversion

Data Type	Bits Acquired
bool	8 bits (1 byte)
char	8 bits (1 byte)
short	16 bits (2 bytes)
int	32 bits (4 bytes)
long long	64 bits (8 bytes)
float	32 bits (4 bytes)
double	64 bits (8 bytes)

This conversion **happens without the need for explicit casting** by the programmer and is done in cases where the conversion is safe and **doesn't result in loss of data or precision.**

Implicit Type Conversion

int -> long long

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int x = 10;
    long long y = x;
    cout << y << endl;
    return 0;
}
```

*The **int** value is **automatically converted** to long long without explicit casting.*

Implicit Type Conversion

int -> long long

C++ 

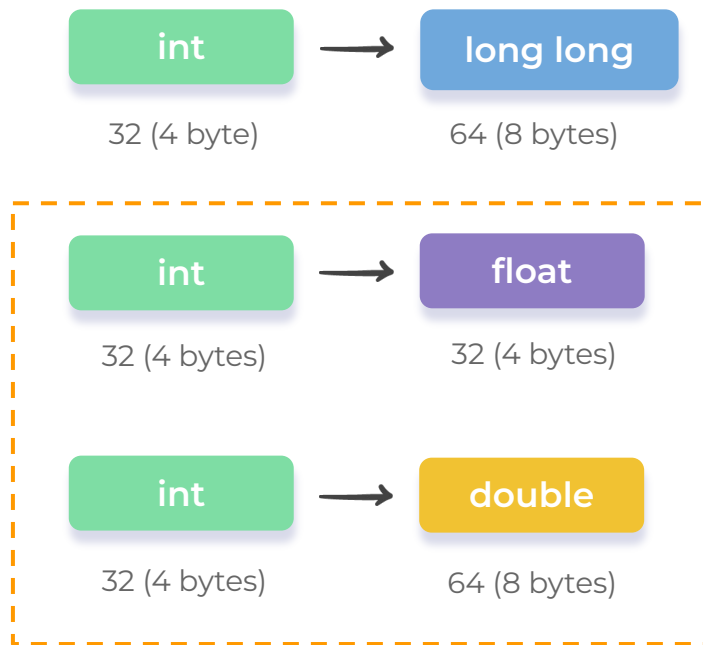
```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int x = 10;
    long long y = x;
    cout << y << endl;
    return 0;
}
```

Output 

10

C++

Implicit Type Conversion



Similarly, `int` can be implicitly converted to `float` or `double`.

Implicit Type Conversion

char → int

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    char ch = 'B';
    int number = ch;
    cout << number << endl;
    return 0;
}
```

Output 

66

short → char

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    short number = 65;
    char ch = number;
    cout << ch << endl;
    return 0;
}
```

Output 

A

Implicit Type Conversion

short → char

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    short number = 300;
    char ch = number;
    cout << ch << endl;
    return 0;
}
```

Output 

,

ASCII Value of comma ',': 44

Binary Representation

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    short number = 300;
    char ch = number;
    cout << ch << endl;
    return 0;
}
```

*Unlike the previous cases, C++ implicitly **converting** an **int** to a **char** leads to **data loss** after a certain range.*

char → int

short → char

Binary Representation of 300 as short (2 byte)



Data loss

The last 1 byte corresponds to char data type
ASCII value of comma - 44 (decimal)

Implicit vs Explicit

Definition

Manual conversion done by the programmer.

Automatic conversion done by the compiler.

Syntax

Uses type casting (type) value.

No additional syntax required.

Data Loss

Controlled by the programmer to avoid unintended data loss.

Possible when converting larger types to smaller types.

Control

Requires explicit casting by the programmer.

Fully handled by the compiler.

Example

```
int x = 10;  
long long y = (long long)x;
```

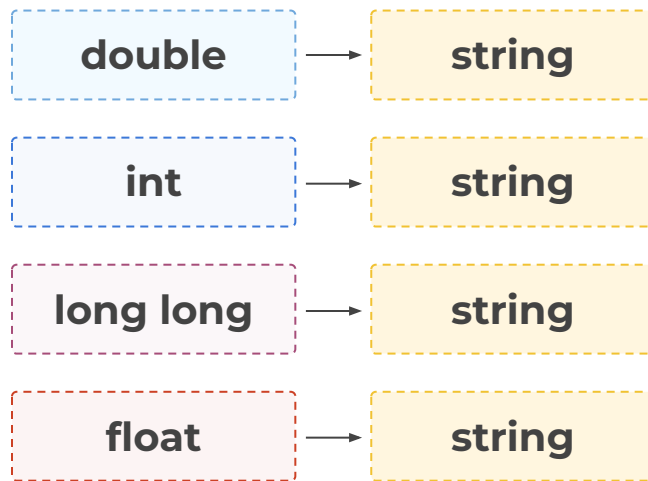
```
int x = 10;  
long long y = x;
```

Type Conversion Methods



Primitive Data Types to String

In C++, we use the `to_string()` function for converting **primitive data types** to **strings**.



Primitive Data Types to String

int → string

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int number = 3;
    string str = to_string(number);
    cout << str << endl;
return 0;
}
```

Output 

3

to_string function

```
short num1 = 10;  
string str = to_string(num1);
```

```
long long num2 = 12345678912;  
string str = to_string(num2);
```

```
float num3 = 10.4;  
string str = to_string(num3);
```

```
double num4 = 5.46;  
string str = to_string(num4);
```

String.valueOf()

char → String

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    char ch = 'a';
    string str(1, ch);
    cout << str << endl;
    return 0;
}
```

Creates a string of length 1 containing the character stored in **ch**, then prints the string using **cout**.

String.valueOf()

char → String

C++ 

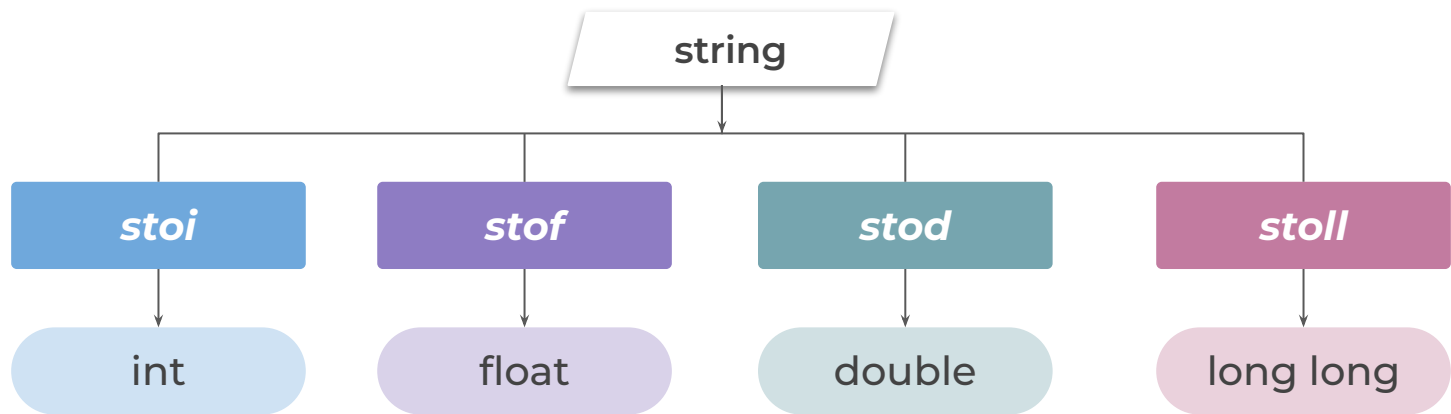
```
#include <bits/stdc++.h>
using namespace std;
int main() {
    char ch = 'a';
    string str(1, ch);
    cout << str << endl;
    return 0;
}
```

Output 

a

String to Primitive

In C++, we use functions like `stoi()`, `stof()`, `stod()`, and `stoll()` to convert strings into primitive data types.



String to Primitive

string → int

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "123";
    int num = stoi(s);
    cout << num << endl;
    return 0;
}
```

Output 

123

String to Primitive

string → double

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "3.14159";
    double d = stod(s);
    cout << d << endl;
    return 0;
}
```

Output 

3.14159

String to Primitive

string → float

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "3.14";
    float f = stof(s);
    cout << f << endl;
    return 0;
}
```

Output 

3.14

String to Primitive

string → long long

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "123456789";
    long long num = stoll(s);
    cout << num << endl;
    return 0;
}
```

Output 

123456789

Quiz Time!



Conditional Statements



v/s



Conditional Statements

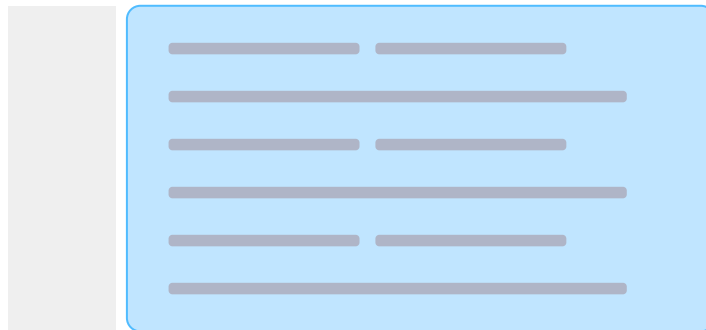
```
if (condition) {
```



```
}
```



```
if condition:
```



if Statement

C++ 

```
if (score < 0) {  
    cout << "Negative" << endl;  
}
```

Python 

```
if score < 0:  
    print("Negative")
```

if Statement

Example

C++ 

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int score = -10;
    if (score < 0) {
        cout << "Negative" << endl;
    }
    return 0;
}
```

Output 

Negative

if-else Statement

C++ 

```
if (score < 0) {  
    cout << "Negative" << endl;  
}  
else {  
    cout << "Positive" << endl;  
}
```

Python 

```
if score < 0:  
    print("Negative")  
else:  
    print("Positive")
```

if-else Statement

Example

C++



```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int score = 25;
    if (score < 0) {
        cout << "Negative" << endl;
    }
    else {
        cout << "Positive" << endl;
    }
    return 0;
}
```

Output



Positive

nested if-else

C++



```
if (score < 0) {  
    cout << "Score is negative" << " ";  
    if (score < -20) {  
        cout << "and score is less than -20." << endl;  
    }  
}  
else {  
    cout << "Score is positive" << " ";  
    if (score > 20) {  
        cout << "and score is greater than 20." << endl;  
    }  
}
```

Python



```
if score < 0:  
    print("Score is negative", end = " ")  
    if score < -20:  
        print("and score is less than -20.")  
else:  
    print("Score is positive", end = " ")  
    if score > 20:  
        print("and score is greater than 20.")
```


Nested if-else

Example

C++



```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int score = 25;
    if (score < 0) {
        cout << "Score is negative" << " ";
        if (score < -20) {
            cout << "and score is less than -20." << endl;
        }
    }
    else {
        cout << "Score is positive" << " ";
        if (score > 20) {
            cout << "and score is greater than 20." << endl;
        }
    }
    return 0;
}
```

Output



**Score is positive and
score is greater than 20**

if-else-if Ladder

C++ 

```
if (score < 0) {  
    cout << "Negative" << endl;  
}  
else if(score > 0){  
    cout<< "Positive" << endl;  
}  
else {  
    cout << "Zero" << endl;  
}
```

Python 

```
if score < 0:  
    print("Negative")  
elif score > 0:  
    print("Positive")  
else:  
    print("Zero")
```

if-else-if Ladder

Example

C++



```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int score = 0;
    if (score < 0) {
        cout << "Negative" << endl;
    }
    else if(score > 0){
        cout<< "Positive" << endl;
    }
    else {
        cout << "Zero" << " ";
    }
    return 0;
}
```

Output



Zero

switch statement

- ✓ A **switch case** in C++ is a conditional control statement that evaluates an **expression** and executes the **matching case block**, using **break** to **prevent fall-through**.

C++



```
int inputValue = 3;
switch (inputValue) {
    case 1:
        cout << "A" << endl; break;
    case 2:
        cout << "B" << endl; break;
    case 3:
        cout << "C" << endl; break;
    default:
        cout << "D" << endl;
}
```

switch statement

Example

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int inputValue = 1;
    switch (inputValue) {
        case 1:
            cout << "A" << endl;
        case 2:
            cout << "B" << endl; break;
        case 3:
            cout << "C" << endl; break;
        default:
            cout << "D" << endl;
    }
    return 0;
}
```

- **break** stops execution after a **matched** case. Without **break**, all cases below the match execute (**fall-through**).
- Use **fall-through** only when needed (e.g., **grouping cases**).

switch statement

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int inputValue = 1;
    switch (inputValue) {
        case 1:
            cout << "A" << endl;
        case 2:
            cout << "B" << endl; break;
        case 3:
            cout << "C" << endl; break;
        default:
            cout << "D" << endl;
    }
    return 0;
}
```

Output 

A

B

Quiz Time!



C++

Key Takeaways

- ✓ ***Operators***

- Logical, Arithmetic, Relational and Assignment operator

- ✓ ***Type Conversion***

- Implicit & explicit type conversion

- ✓ ***Type conversion Method***

- Primitive data types to string & string to primitive.



C++

Key Takeaways

✓ ***Conditional Statements***

→ if, if-else, nested if-else, if-else ladder,
switch statements.

✓ ***Practice Problems***





THANK YOU





ALL THE BEST



Practice Question



Larger of Two Numbers

You're given *two integers*, and
your task is to *find* the *larger* of
the *two numbers*.



Larger of Two Numbers

Code

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    if (a > b) {
        cout << "a is larger" << endl;
    }
    else {
        cout << "b is larger" << endl;
    }
    return 0;
}
```

Input



10 5

Output



a is larger

Python



```
a = int(input())
b = int(input())
if a > b:
    print("a is larger")
else:
    print("b is larger")
```

Input



10 5

Output



a is larger